LAB Manual
PART A
(PART A: TO BE REFFERED BY STUDENTS)

Experiment No.10

A.1 Aim:
Implementation of Page rank/HITS algorithm

A.2 Prerequisite:
Familiarity programming languages.

A.3 Outcome:
After successful completion of this experiment students will be able to

- Explore and Implement web mining

A.4 Theory:

THEORY:

PageRank is a "vote", by all the other pages on the Web, about how important a page is. A link to a page counts as a vote of support. If there's no link there's no support but it's an abstention from voting rather than a vote against the page). Quoting from the original Google paper, PageRank is defined like this:
We assume page A has pages T1...Tn which point to it (i.e., are citations). The parameter d is a damping factor which can be set between 0 and 1. We usually set d to 0.85. There are more details about d in the next section. Also C(A) is defined as the number of links going out of page A. The PageRank of a page A is given as follows:

PR(A) = (1-d) + d (PR(T1)/C(T1) + ... + PR(Tn)/C(Tn))

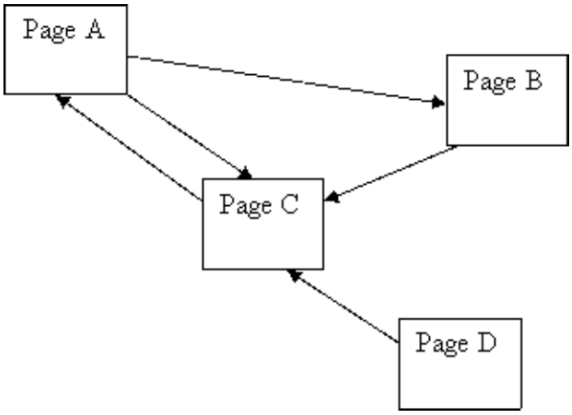
Note that the PageRanks form a probability distribution over web pages, so the sum of all web pages' PageRanks will be one. PageRank or PR(A) can be calculated using a simple iterative algorithm, and corresponds to the principal eigenvector of the normalized link matrix of the web. but that's not too helpful so let's break it down into sections.
1. **PR(Tn)** - Each page has a notion of its own self-importance. That's "PR(T1)" for the first page in the web all the way up to "PR(Tn)" for the last page
2. **C(Tn)** - Each page spreads its vote out evenly amongst all of it's outgoing links. The count, or number, of outgoing links for page 1 is "C(T1)", "C(Tn)" for page n, and so on for all pages.
3. **PR(Tn)/C(Tn)** - so if our page (page A) has a backlink from page "n" the share of the vote page A will get is "PR(Tn)/C(Tn)"
4. **d(...** - All these fractions of votes are added together but, to stop the other pages having too much influence, this total vote is "damped down" by multiplying it by 0.85 (the factor "d")

5. **(1 - d)** - The (1 – d) bit at the beginning is a bit of probability math magic so the "sum of all web pages' PageRanks will be one": it adds in the bit lost by the **d(...**. It also means that if a page has no links to it (no backlinks) even then it will still get a small PR of 0.15 (i.e. 1 – 0.85). (Aside: the Google paper says "the sum of all pages" but they mean the "the normalised sum" – otherwise known as "the average" to you and me.

How is PageRank Calculated?

The PR of each page depends on the PR of the pages pointing to it. But we won't know what PR those pages have until the pages pointing to **them** have their PR calculated and so on… And when you consider that page links can form circles it seems impossible to do this calculation! But actually it's not that bad. Remember this bit of the Google paper:

PageRank or PR(A) can be calculated using a simple iterative algorithm, and corresponds to the principal eigenvector of the normalized link matrix of the web. What that means to us is that we can just go ahead and calculate a page's PR **without knowing the final value of the PR of the other pages**. That seems strange but, basically, each time we run the calculation we're getting a closer estimate of the final value. So all we need to do is remember the each value we calculate and repeat the calculations lots of times until the numbers stop changing much.



Node/page	Out link	Incoming link
A	2	1
B	1	1
C	1	3
D	1	0

PR(A) = 1-0.85 + 0.85(1/1) = 0.15+0.85 =1
PR(B) = 1-085 + 0.85(1/2) = 0.575
PR(C) = 1-0.85 + 0.85(1/2+0.575/1+1/1) = 0.15 + 0.85*2.5 = 1.91375
PR(D) = 0.15 + 0.85*0 = 0.15
PR = C,A,B,D
K=2
PR(A) = 0.15 + 0.85 (1.91375/1) = 1.7766875
PR(B) = 0.15 + 0.85(1.7766875/2) = 0.9050921875
PR(C) = 0.15 + 0.85(1.7766875/2+0.9050921875/1+0.15/1) = 1.8019
PR(D) = 0.15
PR = C,A,B,D

PART B

(PART B: TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded on the Blackboard or emailed to the concerned lab in charge faculties at the end of the practical in case the there is no Black board access available)

Roll. No.64	Name: Aditi Anant Shanbhag
Class: TE A	Batch: A3
Date of Experiment: 6/10/25	Date of Submission:7/10/25
Grade:	

B.1 Software Code written by student:

main.py

```python

import numpy as np

adj_matrix = np.array([...], dtype=float)

n = adj_matrix.shape[0]
d = 0.85
iterations = 100
pagerank = np.ones(n) / n


out_degree = np.sum(adj_matrix, axis=1)


for _ in range(iterations):
    new_rank = np.ones(n) * (1 - d) / n
    for i in range(n):
        for j in range(n):
            if adj_matrix[j, i] == 1:
                new_rank[i] += d * (pagerank[j] / out_degree[j])
    pagerank = new_rank


for i, rank in enumerate(pagerank):
    print(f"Node {chr(65+i)}: {rank:.4f}")

```

B.2 Input and Output:

```
Output                                                    Clear
Node A: 0.3725
Node B: 0.1958
Node C: 0.3941
Node D: 0.0375

=== Code Execution Successful ===
```

B.3 Observations and Learning

PageRank Algorithm

- The output of the PageRank algorithm showed that nodes with a higher number of **incoming links** received a **higher rank score**, indicating their importance in the network.
- Node **C** had the highest PageRank value because most nodes in the graph linked to it, making it a strong "authority" node.
- Node **D** had the lowest rank as it contributed outgoing links but received none.
- The PageRank values gradually stabilized after multiple iterations, demonstrating **convergence** — a key property of the algorithm.
- Learning: The PageRank algorithm mimics the way web pages are ranked on search engines like Google, by assigning numerical weights to each element of a hyperlinked set.

HITS Algorithm

- The HITS algorithm provided two scores for each node: **authority** and **hub**.
- Node **A** acted as a strong **hub** (pointing to other important nodes), while Node **C** acted as a strong **authority** (receiving links from good hubs).

- Normalization ensured the stability and comparability of scores across iterations.
- Learning: The HITS algorithm is effective for topic-specific searches and helps distinguish between nodes that serve as good information sources (hubs) and those that contain valuable information (authorities).
- Both PageRank and HITS highlight how **link structure analysis** can determine the relative importance of nodes in a network graph.

B.4 Conclusion

From the implementation and results of the PageRank and HITS algorithms, we conclude that:

- **PageRank** helps identify globally important nodes within a network by evaluating the number and quality of incoming links. It is particularly useful in large-scale applications such as **search engine ranking**.
- **HITS** distinguishes between **hubs** and **authorities**, providing a more topic-specific understanding of node importance in information networks.
- Both algorithms reinforce the concept that the **structure and interconnection of data** can be analyzed mathematically to determine relevance, importance, and influence.
- The experiment enhanced understanding of **graph-based ranking techniques**, **iterative convergence**, and the **role of link analysis** in data mining and information retrieval systems.

B.5 Questions of Curiosity

Q1: Explain the advantages and disadvantages of agglomeration and hierarchical clustering.

Advantages:

- Does not require pre-specifying the number of clusters.
- Produces a **hierarchical structure (dendrogram)**, which is easy to visualize and interpret.
- Works well for **small datasets** and gives insight into the nested grouping of data.
- Can handle different types of distance or similarity measures.

Disadvantages:

- Computationally expensive for large datasets.
- Once a merge or split is done, it **cannot be undone** (no backtracking).
- Sensitive to **noise and outliers**, which may distort the hierarchy.
- Choosing the right **linkage method** (single, complete, average) affects results significantly.

Q2: What is the relationship between top-down, bottom-up, and division/agglomeration?

- **Bottom-up (Agglomerative)**:
  Starts with each data point as its **own cluster** and **iteratively merges** the most similar

clusters until a single cluster (or desired number) is formed.
➤ It follows a **merge (agglomeration)** process.
Example: Agglomerative Hierarchical Clustering.

- **Top-down (Divisive)**:
  Starts with **all data points in one cluster** and **recursively splits** them into smaller clusters based on dissimilarities.
  ➤ It follows a **division** process.
  Example: Divisive Hierarchical Clustering.

Relationship:
Both approaches create a **hierarchical structure** but in **opposite directions** — bottom-up builds the hierarchy through merging (agglomeration), while top-down builds it through splitting (division).